CSC3060 Bomblab Hints

CSC3060 Teaching Team

(Please read this!)

There are many ways of defusing your bomb. You can examine it in great detail without ever running the program, and figure out exactly what it does. This is a useful technique, but it not always easy to do. You can also run it under a debugger, watch what it does step by step, and use this information to defuse it. This is probably the fastest way of defusing it.

We do make one request, *please do not use brute force!* You could write a program that will try every possible key to find the right one. But this is no good for several reasons:

- You lose 1 point (up to a max of 20 points) every time you guess incorrectly and the bomb explodes.
- Every time you guess wrong, a message is sent to the bomblab server. You could very quickly saturate the network with these messages, and cause the system administrators to revoke your computer access.
- We haven't told you how long the strings are, nor have we told you what characters are in them. Even if you made the (incorrect) assumptions that they all are less than 80 characters long and only contain letters, then you will have 26^{80} guesses for each phase. This will take a very long time to run, and you will not get the answer before the assignment is due.

Hints for every phase:

1. *What can we get from that address?* Refer to tutorial materials and the teaching video.
2. *What's wrong with the stack?* Clarify the logic of assembler code.
3. *Where are we jumping to?* Refer to tutorial materials and the teaching video.
4. *Why this just doesn't stop?* It keeps doing and doing again.
5. *Why is everything encrypted?* You might need to refer to the ASCII code, which is included in figure 1. (Under certain cases)
6. *What's wrong with this mess?* Everything is all linked.

7. *Is this the end?* What secret phase? :-)

Register Usage Table:

Name	ID	Usage	Notes
zero	x0	constant value 0	Invisible to gdb & Cannot be altered
ra	x1	return address	Stores the return address of a function call
sp	x2	stack pointer	Points to the top of the stack, very useful!!
gp	x3	global pointer	Points to global data in the data section
tp	x4	thread pointer	Not used in this lab
t0-t2	x5-x7	temporary registers	Used for temporary storage
s0/fp	x8	saved register / frame pointer	Displayed as fp in gdb but s0 in disassembly
s1	x9	saved register	General purpose saved register
a0-a7	x10-x17	function args / return vals	Pass function arguments and store return values
s2-s11	x18-x27	saved register	General purpose saved register
t3-t6	x28-x31	temporary registers	Additional temporary registers
pc	-	program counter	Where the current instruction is being executed

Questions You May Ask:

Q: ***I don't know where to start. What shall I do?***
A: Go review our tutorial content and practice bomb. The practice bomb is designed for you to solve at least the first four phases, and you should be equipped with the necessary knowledge to solve the rest. Stay composed, do not panic, and believe in yourself!

Q: ***Oh god... Why is this lab so hard? Why is the logic so twisted?***
A: RISC-V, with the R standing for "reduced", follows a minimalist design philosophy. Therefore, unlike x86, the job that one instruction can do is pathetically limited... You will often see instructions doing seemingly nonsense, because several instructions need to combine to perform one task. Additionally, the executable file's instruction layout aims for execution performance, not for human readability. Even the lab designer finds reading the disassembled file a pain in the ass. Trust me, the lab designer has really tried to minimize the lab difficulty. If you still find it hard, blame RISC-V :p

Q: ***Wait... So all we need to write is a simple txt file? No codes?***
A: Yes.

Q: ***But doesn't that make it really easy to cheat?***
A: You bet :-)

Bin Dec Hex

ASCII可显示字符（共95个）

二进制	十进制	十六进制	图形	二进制	十进制	十六进制	图形	二进制	十进制	十六进制	图形
0010 0000	32	20	(space)	0100 0000	64	40	@	0110 0000	96	60	`
0010 0001	33	21	!	0100 0001	65	41	A	0110 0001	97	61	a
0010 0010	34	22	"	0100 0010	66	42	B	0110 0010	98	62	b
0010 0011	35	23	#	0100 0011	67	43	C	0110 0011	99	63	c
0010 0100	36	24	$	0100 0100	68	44	D	0110 0100	100	64	d
0010 0101	37	25	%	0100 0101	69	45	E	0110 0101	101	65	e
0010 0110	38	26	&	0100 0110	70	46	F	0110 0110	102	66	f
0010 0111	39	27	'	0100 0111	71	47	G	0110 0111	103	67	g
0010 1000	40	28	(	0100 1000	72	48	H	0110 1000	104	68	h
0010 1001	41	29	)	0100 1001	73	49	I	0110 1001	105	69	i
0010 1010	42	2A	*	0100 1010	74	4A	J	0110 1010	106	6A	j
0010 1011	43	2B	+	0100 1011	75	4B	K	0110 1011	107	6B	k
0010 1100	44	2C	,	0100 1100	76	4C	L	0110 1100	108	6C	l
0010 1101	45	2D	–	0100 1101	77	4D	M	0110 1101	109	6D	m
0010 1110	46	2E	.	0100 1110	78	4E	N	0110 1110	110	6E	n
0010 1111	47	2F	/	0100 1111	79	4F	O	0110 1111	111	6F	o
0011 0000	48	30	0	0101 0000	80	50	P	0111 0000	112	70	p
0011 0001	49	31	1	0101 0001	81	51	Q	0111 0001	113	71	q
0011 0010	50	32	2	0101 0010	82	52	R	0111 0010	114	72	r
0011 0011	51	33	3	0101 0011	83	53	S	0111 0011	115	73	s
0011 0100	52	34	4	0101 0100	84	54	T	0111 0100	116	74	t
0011 0101	53	35	5	0101 0101	85	55	U	0111 0101	117	75	u
0011 0110	54	36	6	0101 0110	86	56	V	0111 0110	118	76	v
0011 0111	55	37	7	0101 0111	87	57	W	0111 0111	119	77	w
0011 1000	56	38	8	0101 1000	88	58	X	0111 1000	120	78	x
0011 1001	57	39	9	0101 1001	89	59	Y	0111 1001	121	79	y
0011 1010	58	3A	:	0101 1010	90	5A	Z	0111 1010	122	7A	z
0011 1011	59	3B	;	0101 1011	91	5B	[	0111 1011	123	7B	{
0011 1100	60	3C	<	0101 1100	92	5C	\	0111 1100	124	7C	|
0011 1101	61	3D	=	0101 1101	93	5D	]	0111 1101	125	7D	}
0011 1110	62	3E	>	0101 1110	94	5E	^	0111 1110	126	7E	~
0011 1111	63	3F	?	0101 1111	95	5F	_				

Figure 1: This is ASCII.